

Documentation Technique - VoltIQ

Ce document fournit une explication détaillée de l'architecture, du fonctionnement et de la logique métier de l'application **VoltIQ**.

1. Vue d'ensemble du Projet

VoltIQ est une application web progressive (PWA) conçue pour aider les utilisateurs de compteurs prépayés (comme la CIE en Côte d'Ivoire) à suivre leur consommation d'énergie, anticiper les coupures et visualiser leur historique de recharges.

- **Frontend** : React.js (créé avec Vite)
 - **Routage** : React Router v6
 - **Gestion d'état** : React Context API (**DataContext**)
 - **Stockage des données** : **localStorage** (Les données sont sauvegardées localement dans le navigateur de l'utilisateur, aucune base de données distante n'est utilisée pour le moment).
 - **Style** : CSS natif (**index.css**) et variables CSS pour le thème (support du mode sombre et clair).
-

2. Architecture des Fichiers

L'application est structurée de manière modulaire dans le dossier **src/** :

```
src/
├── assets/           # Images, icônes et logos (SVG, PNG)
├── components/       # Composants réutilisables
│   ├── ErrorBoundary.jsx # Capture d'erreurs pour éviter le plantage
│   ├── Layout.jsx       # Mise en page principale (barre de navigation, logo)
│   └── Onboarding.jsx    # Écran de bienvenue affiché à la première utilisation
├── context/
│   └── DataContext.jsx   # Gestion globale de l'état (historique, paramètres)
├── pages/            # Pages principales (affichées via le routeur)
│   ├── Dashboard.jsx    # Tableau de bord (Alerte, Jours restants, Graphes)
│   ├── History.jsx      # Liste de l'historique des recharges
│   ├── NewTopUp.jsx     # Formulaire pour ajouter une nouvelle recharge
│   └── Settings.jsx      # Paramètres et gestion des notifications
├── utils/
│   └── calculations.js   # Logique métier pure (calculs de consommation)
├── App.jsx           # Configuration des routes (React Router)
├── index.css          # Styles globaux (couleurs, boutons, cartes)
└── main.jsx           # Point d'entrée de l'application React
```

3. Logique et Gestion des Données (**DataContext.jsx**)

Le cœur de l'application réside dans **DataContext.jsx**. Il utilise l'API Context de React pour rendre les données accessibles à n'importe quel composant de l'application sans avoir à passer des **props** en cascade.

Les Données Stockées (State) :

- **recharges** : Un tableau contenant l'historique complet des recharges.
- **currentRecharge** : Un objet contenant les informations de la recharge active (celle en cours d'utilisation).
- **hasCompletedOnboarding** : Un booléen qui indique si l'utilisateur a déjà passé l'écran de bienvenue (Onboarding).

Le cycle de vie des données :

1. **Initialisation** : Au démarrage, l'application lit le **localStorage** pour récupérer l'état précédent.
2. **Ajout d'une recharge** : Lorsqu'une nouvelle recharge est ajoutée, la fonction **addRecharge** est appelée. Elle utilise la fonction utilitaire **calculateRechargeData** pour faire les estimations.
3. **Persistence** : À chaque fois que **recharges** ou **hasCompletedOnboarding** change, un **useEffect** se déclenche et met à jour le **localStorage**.

4. Les Calculs de Consommation (**calculations.js**)

Le fichier **src/utils/calculations.js** est responsable de toute l'intelligence de prédiction de l'application.

Comment ça marche ?

Lorsqu'un utilisateur ajoute une nouvelle recharge, l'application a besoin de calculer :

1. **L'énergie totale disponible** : C'est la somme de la nouvelle énergie achetée + l'énergie restante déclarée sur le compteur. **totalEnergy = newEnergy + remainingEnergy**
2. **La consommation moyenne (si applicable)** : Si l'utilisateur a une recharge précédente dans l'historique, l'application calcule combien de temps s'est écoulé depuis la dernière recharge et combien d'énergie a été consommée. **consumedEnergy = prevTotalEnergy - remainingEnergy**
avgConsumption = consumedEnergy / daysElapsed
3. **La durée estimée de la nouvelle recharge** : En utilisant la consommation moyenne calculée, on peut estimer combien de jours durera la nouvelle recharge. **estimatedDuration = totalEnergy / avgConsumption**
4. **La date d'épuisement prévue** : On ajoute la **estimatedDuration** à la date de la recharge pour trouver la date de fin.

Remarque : Lors de la toute première recharge, la consommation moyenne ne peut pas être calculée. Elle est définie à 0.

5. Les Pages et Composants Clés

App.jsx

Le fichier **App.jsx** gère la navigation. Si **hasCompletedOnboarding** est faux, il force l'affichage du composant **<Onboarding />**. Sinon, il affiche les pages normales via **<Routes>**.

Dashboard.jsx (Le Tableau de Bord)

Il affiche le statut actuel :

- **Alerte** : Si le nombre de jours restants est ≤ 3 , une alerte s'affiche.
- **Carte restante** : Une barre de progression à 7 segments représentant le pourcentage d'énergie restant par rapport à l'estimation initiale.
- **Graphe historique** : Un petit graphique en barres généré dynamiquement à partir des 9 dernières consommations moyennes.

NewTopUp.jsx (Nouvelle Recharge)

Un formulaire en deux étapes :

1. **Étape 1** : L'utilisateur saisit ses données (Montant, Énergie, Reste, Date). Le composant fait un calcul préliminaire (similaire à `calculations.js`) pour afficher une preview.
2. **Étape 2** : L'utilisateur voit l'estimation ("Votre recharge devrait durer X jours") et peut valider pour l'ajouter à la base de données.

6. L'UI et le Design (`index.css`)

Le design a été conçu pour être "Mobile-First" (adapté principalement pour les téléphones). Il n'utilise pas de librairie CSS comme Tailwind ou Bootstrap. Tout est géré dans `index.css` via des variables CSS (`--color-primary`, `--color-bg-main`, etc.).

Cela permet de changer très facilement les couleurs globales et supporte automatiquement le Mode Sombre (`@media (prefers-color-scheme: dark)`).

- Les boutons utilisent la classe `.btn`.
- Les blocs de contenu utilisent la classe `.card` (fond blanc/sombre avec une légère ombre).
- Les champs de saisie sont englobés dans `.input-group`.

7. Déploiement et PWA

L'application est déployée sur **Vercel** via l'intégration GitHub. Elle possède un fichier `manifest.webmanifest` (dans le dossier `public/`) qui permet aux utilisateurs de l'installer sur leur smartphone comme une application native (Ajouter à l'écran d'accueil).